

Gestion des diplômes

SICORE — Fiche technique, API JSON et support de maintenance

Interface administrateur → Contrôleur Front → API REST sécurisée → Contrôleur Back → Base de données

1. Objectif du module

Le module Diplômes est un référentiel de paramétrage. Il permet aux administrateurs de consulter, rechercher, ajouter, modifier et supprimer les diplômes utilisés dans SICORE, notamment pour les informations des enseignants.

2. Architecture et sécurité

1. L'utilisateur agit depuis la vue `parametres/diplomes`.
2. Le contrôleur du front valide les données et appelle le back par HTTP.
3. Le jeton de connexion est transmis dans l'en-tête `Authorization: Bearer ....`
4. L'API vérifie l'authentification Sanctum et le rôle.
5. Les contrôleurs et Form Requests du back appliquent les règles métier.
6. Le modèle Eloquent lit ou modifie la table `diplomes`.
7. Une ressource Laravel normalise le JSON retourné au front.

Autorisation : le middleware du front `diplomes.manage` et l'API `auth:sanctum + role:admin,super_admin` réservent la gestion aux Administrateurs et Super Administrateurs.

3. Structure d'un diplôme

Champ	Rôle	Règle
<code>id</code>	Identifiant technique	Généré automatiquement
<code>code</code>	Code court du diplôme	Obligatoire, 20 caractères maximum, unique
<code>libelle</code>	Nom complet	Obligatoire, 100 caractères maximum
<code>type</code>	Catégorie	Optionnel : <code>academique</code> ou <code>professionnel</code>
<code>date_obteention</code>	Date d'obtention	Date valide, obligatoire dans les formulaires

Attention pour la maintenance : le code utilise partout le nom `date_obteention` avec deux « e ». C'est cohérent entre le front, l'API et la base, mais le nom devrait idéalement être corrigé en `date_obtention` par une migration contrôlée.

4. Flow de consultation

Quand l'administrateur ouvre la page, le front construit les paramètres `search`, `type`, `page` et `per_page=15`, puis appelle :

```
GET /api/diplomes?search=master&type=academique&page=1&per_page=15
Authorization: Bearer <access_token>
Accept: application/json
```

Le back recherche le texte dans les colonnes `code` et `libelle`, filtre éventuellement par type, trie les résultats par libellé puis applique la pagination.

Réponse HTTP 200

```
{
  "message": "Liste des diplômes",
  "data": [
    {
      "id": 1,
      "code": "MASTER",
      "libelle": "Master en informatique",
      "type": "academique",
      "date_obteention": "15/07/2025",
      "created_at": "2026-08-20T10:00:00+00:00",
      "updated_at": "2026-08-20T10:00:00+00:00"
    }
  ],
  "meta": {
    "current_page": 1,
    "last_page": 1,
    "per_page": 15,
    "total": 1
  }
}
```

5. Flow d'ajout

1. L'administrateur ouvre la fenêtre « Ajouter un diplôme ».
2. Il renseigne le code, le libellé, le type et la date.
3. Le front valide les champs puis appelle **POST /api/diplomes**.
4. Le back refait la validation, notamment l'unicité du code.
5. `Diplome::create()` insère le diplôme.
6. L'API retourne HTTP 201 et le diplôme créé.

JSON envoyé

```
{
  "code": "MASTER",
  "libelle": "Master en informatique",
  "type": "academique",
  "date_obteention": "2025-07-15"
}
```

Réponse HTTP 201

```
{
  "message": "Diplôme créé avec succès",
  "data": {
    "id": 1,
    "code": "MASTER",
    "libelle": "Master en informatique",
    "type": "academique",
    "date_obteention": "15/07/2025",
    "created_at": "2026-08-20T10:00:00+00:00",
    "updated_at": "2026-08-20T10:00:00+00:00"
  }
}
```

Exemple d'erreur HTTP 422

```
{
  "message": "The given data was invalid.",
  "errors": {
    "code": ["Ce code de diplôme existe déjà."]
  }
}
```

6. Flow d'affichage d'un détail

```
GET /api/diplomes/1
Authorization: Bearer <access_token>
```

```
{
  "message": "Détail du diplôme",
  "data": {
    "id": 1,
    "code": "MASTER",
```

```
"libelle": "Master en informatique",
"type": "academique",
"date_obteention": "15/07/2025",
"created_at": "2026-08-20T10:00:00+00:00",
"updated_at": "2026-08-20T10:00:00+00:00"
}
}
```

7. Flow de modification

1. Le bouton Modifier fournit à la modale les valeurs actuelles du diplôme.
2. L'administrateur change les informations.
3. Le front appelle **PUT /api/diplomes/{id}**.
4. Le back valide les valeurs. Pour l'unicité, le code du diplôme actuel est ignoré.
5. Le modèle exécute la mise à jour et l'API retourne la ressource actualisée.

JSON envoyé

```
{
  "code": "MASTER-INFO",
  "libelle": "Master professionnel en informatique",
  "type": "professionnel",
  "date_obteention": "2025-07-15"
}
```

Réponse HTTP 200

```
{
  "message": "Diplôme mis à jour avec succès",
  "data": {
    "id": 1,
    "code": "MASTER-INFO",
    "libelle": "Master professionnel en informatique",
    "type": "professionnel",
    "date_obteention": "15/07/2025",
    "created_at": "2026-08-20T10:00:00+00:00",
    "updated_at": "2026-08-26T14:30:00+00:00"
  }
}
```

8. Flow de suppression

1. L'administrateur clique sur Supprimer.
2. Une fenêtre demande confirmation, car l'action est irréversible.
3. Le front appelle **DELETE /api/diplomes/{id}**.
4. Laravel retrouve automatiquement le diplôme grâce au route model binding.
5. Le back appelle `delete()` et retourne un message de succès.

```
DELETE /api/diplomes/1
Authorization: Bearer <access_token>
```

```
{
  "message": "Diplôme supprimé avec succès"
}
```

9. Résumé des API

Méthode	Route	Fonction	Succès
GET	/api/diplomes	Lister, rechercher et paginer	200
POST	/api/diplomes	Créer	201
GET	/api/diplomes/{id}	Afficher un détail	200
PUT/PATCH	/api/diplomes/{id}	Modifier	200
DELETE	/api/diplomes/{id}	Supprimer	200

10. Gestion des erreurs

- **401** : jeton absent ou utilisateur non authentifié.
- **403** : utilisateur authentifié, mais rôle non autorisé.
- **404** : identifiant du diplôme introuvable.
- **422** : données invalides, code déjà utilisé ou type incorrect.
- **Service indisponible** : le front intercepte l'exception et affiche un message compréhensible.

11. Discours prêt pour la soutenance

« Le module Diplômes est un CRUD de paramétrage. L'administrateur utilise une interface qui communique avec le back à travers une API REST sécurisée. Pour la consultation, le front envoie les paramètres de recherche, de type et de pagination à l'API. Pour l'ajout et la modification, les données sont validées une première fois par le front, puis une seconde fois par le back. Le code doit être unique, le libellé est obligatoire, le type est académique ou professionnel et la date doit être valide. Le back utilise Eloquent pour accéder à la table diplômes et une ressource Laravel pour

normaliser les réponses JSON. La suppression demande une confirmation avant l'appel DELETE. Toutes les requêtes transmettent le bearer token, et seuls les administrateurs ou super administrateurs peuvent gérer ce référentiel. »

12. Questions possibles du jury

Pourquoi valider au front et au back ?

Le front améliore l'expérience utilisateur, tandis que le back constitue la validation de sécurité, car une API peut être appelée sans passer par l'interface.

Pourquoi utiliser une API Resource ?

Pour garantir une structure JSON stable et contrôler précisément les données exposées.

Comment l'unicité du code est-elle garantie ?

Par la validation Laravel et par une contrainte unique dans la base de données.

À quoi sert le route model binding ?

Laravel transforme automatiquement l'identifiant de la route en modèle Diplome et retourne une erreur 404 s'il n'existe pas.

Améliorations possibles : corriger date_obteention, harmoniser le caractère obligatoire de la date entre la migration et la validation, et vérifier les contraintes de relation avant une suppression si le diplôme est déjà associé à un enseignant.